

Pertemuan 1

Pengenalan Struktur Data

A. Tujuan

1. Mahasiswa memahami yang dimaksud dengan struktur data
2. Mahasiswa memahami array, struktur, pointer dalam bahasa C

B. Dasar Teori

Struktur data adalah sebuah skema organisasi, seperti struktur dan array, yang diterapkan pada data sehingga data dapat diinterpretasikan dan sehingga operasi-operasi spesifik dapat dilaksanakan pada data tersebut :

B.1. Array

Array adalah organisasi kumpulan data homogen yang ukuran atau jumlah elemen maksimumnya telah diketahui dari awal. Array umumnya disimpan di memori komputer secara kontigu (berurutan). Deklarasi dari array adalah sebagai berikut:

```
int DATA[10];
```

Artinya variabel DATA adalah kumpulan data sebanyak 10 bilangan bertipe integer. Operasi terhadap elemen di array dilakukan dengan pengaksesan langsung. Nilai di masing-masing posisi elemen dapat diambil dan nilai dapat disimpan tanpa melewati posisi-posisi lain. Terdapat dua tipe operasi, yaitu:

- Operasi terhadap satu elemen/posisi dari array
- Operasi terhadap array sebagai keseluruhan

Dua operasi paling dasar terhadap satu elemen/posisi adalah

- Penyimpanan nilai elemen ke posisi tertentu di array
- Pengambilan nilai elemen dari posisi tertentu di array

Contoh:

`data[10] = 78`, berarti penyimpanan nilai 78 ke posisi ke-10 dari array data

`nilai = data[10]`, berarti pengambilan nilai elemen posisi ke-10 dari array data

Keunggulan array adalah sebagai berikut:

1. Array sangat cocok untuk pengaksesan acak. Sembarang elemen di array dapat diacu secara langsung tanpa melalui elemen-elemen lain.

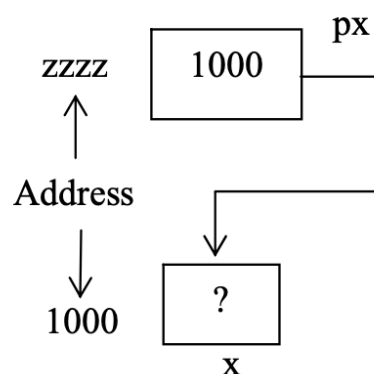
2. Jika berada di suatu lokasi elemen, maka sangat mudah menelusuri ke elemen-elemen tetangga, baik elemen pendahulu atau elemen penerus
3. Jika elemen-elemen array adalah nilai-nilai independen dan seluruhnya harus terjaga, maka penggunaan penyimpanannya sangat efisien

Kelemahan array adalah sebagai berikut:

1. Array mempunyai fleksibilitas rendah, karena array mempunyai batasan sebagai berikut:
2. Array harus bertipe homogen. Kita tidak dapat mempunyai array dimana satu elemen adalah karakter, elemen lain bilangan, dan elemen lain adalah tipe-tipe lain
3. Kebanyakan bahasa pemrograman mengimplementasikan **array statik yang sulit diubah ukurannya di waktu eksekusi**. Bila penambahan dan pengurangan terjadi terus-menerus, maka representasi statis
 - Tidak efisien dalam penggunaan memori
 - Menyiakan banyak waktu komputasi
 - Pada suatu aplikasi, representasi statis tidak dimungkinkan

B.2. Pointer

Variabel pointer sering dikatakan sebagai variabel yang menunjuk ke obyek lain. Pada kenyataan yang sebenarnya, **variabel pointer berisi alamat dari suatu obyek lain** (yaitu obyek yang dikatakan ditunjuk oleh pointer). Sebagai contoh, **px** adalah variabel pointer dan **x** adalah variabel yang ditunjuk oleh **px**. Kalau **x** berada pada alamat memori (alamat awal) 1000, maka **px** akan berisi 1000. Sebagaimana diilustrasikan pada gambar di bawah ini



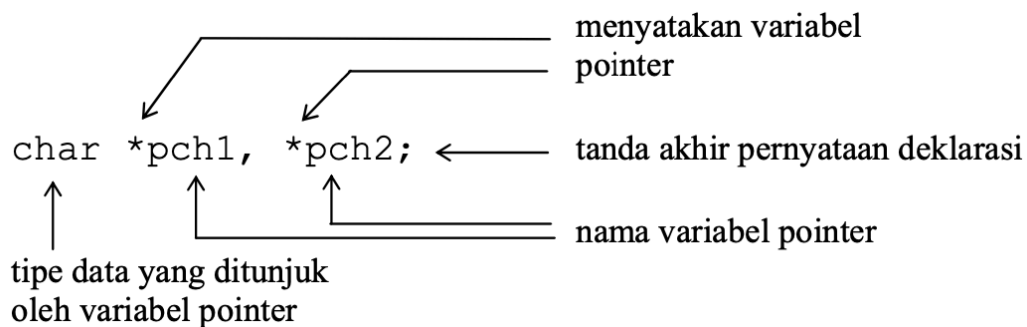
Suatu variabel pointer dideklarasikan dengan bentuk sebagai berikut

```
tipe *nama_variabel
```

dengan **tipe** dapat berupa sembarang tipe data dalam bahasa C. Adapun **nama_variabel** adalah nama dari variabel pointer. Sebagai contoh :

```
int *px;
char *pch1, *pch2;
```

Contoh tersebut menyatakan bahwa **px** adalah variabel pointer yang menunjuk ke suatu data bertipe *int*, sedangkan contoh kedua masing **pch1** dan **pch2** adalah variabel pointer yang menunjuk ke data bertipe *char*.



Agar suatu pointer menunjuk ke variabel lain, mula-mula pointer harus diisi dengan alamat dari variabel yang akan ditunjuk. Untuk menyatakan alamat dari suatu variabel, dengan menempatkan operator **&** (operator alamat) di depan nama variabel. Sebagai contoh

```
&x
```

`&x` berarti/bernilai “alamat dari variabel **x**”. Adapun contoh pemberian alamat **x** ke suatu variabel pointer **px**, yaitu :

```
px = &x;
```

Pernyataan di atas berarti bahwa **px** diberi nilai berupa alamat dari variabel **x**. Jika suatu variabel sudah ditunjuk oleh pointer, variabel yang ditunjuk oleh pointer tersebut dapat diakses melalui variabel itu sendiri (pengaksesan langsung) ataupun melalui pointer (pengaksesan tak langsung). Pengaksesan tak langsung dilakukan dengan menggunakan operator *indirection* (tak langsung) berupa simbol *****. Contoh penerapan operator ***** yaitu :

```
*px
```

yang menyatakan “isi atau nilai variabel/data yang ditunjuk oleh pointer **px**” . Sebagai contoh jika **y** bertipe *int*, maka sesudah dua pernyataan berikut

```
px = &x;
```

```
y = *px;
```

y akan berisi nilai yang sama dengan nilai **x**.

B.3. Struktur

Struktur adalah koleksi dari variabel yang dinyatakan dengan sebuah nama, dengan sifat setiap variabel dapat memiliki tipe yang berlainan. Struktur biasa dipakai untuk mengelompokkan beberapa informasi yang berkaitan menjadi sebuah satu kesatuan. Bentuk umum dalam mendefinisikan dan mendeklarasikan struktur adalah sebagai berikut :

```
struct nama_tipe_struktur
{
    tipe field1;
    tipe field2;
    .
    .
    tipe fieldn;
}variabel_struktur1, ... , variabel_strukturM;
```

```
int data
float data 2
char data 3

struct bagus
{
    char nama;
    int nip;
    float nilai;

    bagus data4;

    data4.nama = "r"; }
    data4.nip = 10;
    data4.nilai = 3.2;
```

Contoh sebuah struktur adalah informasi data tanggal, yang berisi: tanggal, bulan dan tahun. Pendefinisian tipe struktur adalah sebagai berikut:

```
struct data_tanggal
{
    int tanggal;
    int bulan;
    int tahun;
}tgl_lahir;
```

Masing-masing tipe dari elemen struktur dapat berlainan. Adapun variabel_struktur1 sampai dengan variabel_strukturM menyatakan bahwa variabel struktur yang dideklarasikan bisa lebih dari satu. Jika ada lebih dari satu variabel, antara variabel struktur dipisahkan dengan tanda koma. Elemen dari struktur dapat diakses dengan menggunakan bentuk

```
variabel_struktur.nama_field
```

Antara variabel_struktur dan nama_field dipisahkan dengan operator titik (disebut operator anggota struktur). Contoh berikut merupakan instruksi untuk mengisi data pada field tanggal

```
tgl_lahir.tanggal = 30;
```

C. Kesimpulan

1. Struktur data adalah sebuah skema organisasi yang diterapkan pada data sehingga data dapat diinterpretasikan dan sehingga operasi-operasi spesifik dapat dilaksanakan pada data tersebut
2. Apabila kita membuat program dengan data yang sudah kita ketahui batasnya, maka kita bisa menggunakan array (tipe data statis),
3. Untuk sekumpulan data dengan tipe data yang berlainan, namun merupakan satu-kesatuan, kita dapat menggunakan struktur untuk merepresentasikannya

D. Latihan

1. Deklarasikan sebuah variabel array (misalkan nama variabelnya = data) yang mengalokasikan untuk 10 elemen bertipe int/double dan masukkan nilai secara acak pada 10 elemen array tersebut.
2. Tuliskan program untuk memasukkan Nilai NIM anda pada elemen yang ke-lima dari array data.
3. Tuliskan program (gunakan loop for) untuk menghitung nilai akumulasi (total) dari seluruh isi sebuah array tersebut
4. Tuliskan sebuah program dengan mendefinisikan sebuah fungsi `rotasi()` yang menerima tiga parameter berupa variabel a, b, dan c. Fungsi ini melakukan rotasi sehingga nilai a berpindah ke b, b ke c dan nilai c ke a sekembalinya ke fungsi `main()`.
5. Definisikan sebuah struktur (misalkan namanya = date) yang memiliki 3 field bertipe int (misalkan namanya = day, month dan year). Kemudian tuliskan potongan program untuk memasukkan 5 buah tanggal lahir anda dan teman satu kelas sehingga data disimpan dalam sebuah array struktur yang bertipe date.